Leveraging ARM Fast Interrupt Requests within the Linux UIO Framework

Emily Seebeck

Tuesday 14th April, 2026

1 Introduction

In this thesis, we want to have a look at the standard kernel User I/O framework and how to integrate it with the FIQ Interrupt on the ARM Architecture. The FIQ Interrupt is of particular note as it is "reserved for a single, high-priority interrupt source that requires a guaranteed fast response time"[1].

This property is achieved in two steps: Firstly, the FIQ interrupt has a higher priority than IRQ and won't be disabled when handling other exceptions: "All exceptions disable IRQ, only FIQ and reset disable FIQ"[1]. Secondly, the ARM Architecture provides an additional 5 banked registers specific to the FIQ mode, allowing us to retain 40 bytes of memory across interrupts. Moreover, the FIQ entry in the Interrupt Vector Table (IVT) is the last one, making it possible to place the handler code directly afterward and skipping a branch instruction and the associated delay. With all these tricks, we are able to shave off precious processing time and decrease latency. In Chapter TODO, we will be measuring the concrete performance difference between FIQ and IRQ with the same driver code, just with swapped out interrupt implementations.

One important caveat of the FIQ is, that the handler is not able to generate executions such as a supervisor calls (SVC). This however integrates very nicely with the UIO framework, as "only a small kernel module is needed for handling the interrupt", reducing the likelihood of needing any SVCs. Once the interrupt arrives in userspace, the ability to call SVCs is restored, so we won't be restricted by this limitation. Further, the FIQ interrupt is not typically used by Linux making it ideal to map the single interrupt source into userspace and increase performance for one specific userspace application.

[1] https://developer.arm.com/documentation/den0013/d/Exception-Handling/Exception-priorities/FIQ-and-IRQ

With more and more drivers in the Linux Kernel switching to userland, we want to investigate how to improve performance with these drivers.

Writing drivers in userland has numerous advantages such as

- Wrong code won't crash the entire system
- No kernel invariants can be messed with
- Faster Debugging
- Easier Devolopment

On the contrary however, these drivers also suffer from a few caveats introduced by not having the code run in kernel space:

- Performance overhead due to context switches
- Memory Limitations
- Userspace threads might be swapped out
- During memory pressure, the kernel might decide to kill the userspace driver
- Application restart: If the application is not given the opportunity to clean up or crashes, the device might be left in an inconsistent state

In this Thesis, we will primarily focus on the Mini-UART (`UART1`) found on the bcm2711 chip. This is because of two reasons: Firstly, it contains shallow FIFOs (8 bytes) that have to be read by the CPU and can't benefit from hardware acceleration or DMA. Secondly, it can run up to 32 MegaBaud.

2 Setup

Setup between raspis

Kernel Version

3 Linux UIO

UIO is a framework developed for ...

> *For many types of devices, creating a Linux kernel driver is overkill. All that is really needed is some way to handle an interrupt and provide access to the memory space of the device. [...] Hardware that is ideally suited for an UIO driver fulfills all the following:*
>
> - *The device has memory that can be mapped. The device can be controlled completely by writing to this memory.*
> - *The device usually generates interrupts.*
> - *The device does not fit into one of the standard kernel subsystems.*

This fits very well for our usecase, the Mini-UART of the Raspberry Pi 4b, in particular because the device "has shallow FIFOs, no DMA Support", can be controlled via Memory-Mapped-IO, meaning a simple `mmap` will suffice and it generates interrupts.

3.1 UIO Basics

The following is a excerpt from the Linux UIO HowTo:

Each UIO device is accessed through it's corresponding device file, `/dev/uioX`, which we'll call `/dev/uio0`.

Memory of the device can be accessed with a call to `mmap(/dev/uio0)`. Multiple mappings are also possible, and can be distinguished with the `offset` parameter of `mmap`. We will, however, only be focusing on a single mapping as the Mini-UART can be controlled with a single address space.

The Userspace part of the driver can wait for interrupts by calling `read(/dev/uio0)`. It will return as soon as an interrupt occurs and return the total interrupt count since inserting the kernel module. This integer is monotonically increasing and, if no interrupts are missed, should always do so by 1.

UIO also implements a `write(/dev/uio0)` function which will call `irqcontrol()`. Writing a 0 disables interrupts and 1 enables them. This is intended for situations where userspace cannot determine what the interrupt source was if the interrupt is already acknowledged. In this case, the kernel can disable interrupts completely and let userspace figure out what has happened.

We will be using this approach for interrupt handling due to the abysmal interrupt interface given by broadcom.

4 Writing the Kernel Module

With the example `uio_cif.c` example it is very straightforward to figure out how to write the Kernel Module.

First, we have to unregister the default serial driver for the Mini-UART. The default driver defines the `compatible` string to be `brcm,bcm2835-aux-uart`.

We overwrite the `compatible` string in the device tree file to

```
&uart1 {
    compatible = "uio_uart";
    ...
}
```

With this change, the default driver won't register, and we are free to provide our own.

Probing the device is quite simple as well. To register a UIO Device we must fill the `struct uio_info`. We provide the handle to the custom `irqcontrol()` function here and an implementation of `open()` so interrupts are only enabled once the device is used. The corresponding `release()` function disables interrupts once the driver stops using the device file.

For memory, we will have a look at the `pdev->resource[0]`. It contains all needed information to populate `struct uio_mem` and is given by the dts. We align the memory to the page boundary, meaning it will start from `0x07e215000` and not `0x07e215040`. why?

Local state can be maintained through the `priv` pointer in the `uio_info struct`. We use it to keep track of enabled / disabled interrupts.

4.1 The Interrupt Handler

Our hardware needs *some* action to be performed after each interrupt: Either acknowledging it by reading the byte or disabling interrupts from the hardware. This limitation is imposed by the hardware as "The Interrupt Line is asserted whenever the Receive FIFO holds at least 1 byte / Transmit FIFO is empty".

If we were not to do anything, the interrupt would be asserted before our userspace driver is scheduled and could handle the byte leading to our kernel interrupt handler being called all the time.

We present two approaches to handle this limitation: Disabling Interrupts and using the `SCRATCH` register.

4.1.1 Disabling Interrupts

This approach is the most straightforward: Simply disable receive / transmit interrupts once one occurs and return `IRQ_HANDLED`. This will give the userspace part of the driver enough time to be scheduled and read the `IO` register. Once the byte has been read, it enables interrupts by writing a `1` to `/dev/uio0`.

The following shows a flow graph of the interrupt handling routine

Figure 1: Interrupt Handling when disabling Interrupts

4.1.2 The `SCRATCH` Register

The second approach we want to present is using the `SCRATCH` register. It "is a single byte of temporary storage" provided by the UART. The Kernel will, upon entering the interrupt handling routine, read the available byte from the `IO` register and save it to the `SCRATCH` register.

A limitation imposed by this approach is that we are only ever able to handle a single byte per interrupt. With the previous approach, the userspace could read check the `AUX_MU_STAT_REG` register for the FIFO level and read as many bytes as there are available. With this approach, the kernel needs to place the byte from the `IO` register into the `SCRATCH` register, meaning a context switch into kernel-space is necessary for every byte.

The following shows a flow graph of the interrupt handling routine

Figure 2: Interrupt Handling when buffering in the `SCRATCH` register

As we first step, we provide a reference implementation with polling. This is the baseline in terms of performance as the CPU speed will be the bottleneck for data throughput. Next, we want to investigate the improvements we will be able to achieve with interrupts, specifically IRQ. Finally, we provide a patch for the Linux Kernel enabling the use of FIQ in UIO. The differences in latency and throughput will be of great interest.

4.2 Enabling the UART

Adding the following lines to `/boot/firmware/config.txt` enables the mini-UART:

```
enable_uart=1
core_freq=250
```

Additionally, the linux console has to be disabled with the `raspi-config` binary.[2] Of course, the custom dtb has to be booted by the kernel to recognize the mini-uart as a uio device

4.3 Measurements

- How to measure?

4.4 Details

Second Raspi

- NFS Booting

[2] https://www.raspberrypi.com/documentation/computers/configuration.html#disabling-the-linux-serial-console